

The General Theory of State Change

The Normalization of Behavior

Registry: [\[HOWL-INFO-17-2026\]](#)

Series Path: [\[HOWL-COMP-1-2026\]](#) → ... → [\[HOWL-COMP-12-2026\]](#) → [\[HOWL-INFO-11-2026\]](#) → ... → [\[HOWL-INFO-13-2026\]](#) → [\[HOWL-MATH-15-2026\]](#) → ... → [\[HOWL-MATH-20-2026\]](#) → [\[HOWL-INFO-14-2026\]](#) → [\[HOWL-INFO-15-2026\]](#) → [\[HOWL-INFO-16-2026\]](#) → [\[HOWL-INFO-17-2026\]](#)

Date: August 2026

DOI: 10.5281/zenodo.22493091

Domain: Information Theory / Information Processing Theory

AI Usage Disclosure: Only the top metadata, figures, refs and final copyright sections were edited by the author. All paper content was LLM-generated using Anthropic's Claude Opus 4.6.

Abstract

Codd normalized the form of data. He identified what all stored information has in common, made that common structure the only structure, and stated falsifiable rules that test whether a system genuinely realizes the model. The behavioral half of computing never received the equivalent treatment. Applications still consist of bespoke verbs: hand-written functions that each privately validate, mutate, and log state. The costs practitioners treat as inevitable — duplicated validation, integration defects between features, unauditible state, decision layers that disagree with the rules they act under — are all costs of verbs being many, private, and mutually ignorant.

This paper states the missing symmetric theory: a normalization of behavior. The claim divides into three parts of different kinds. The principles are theory: twelve statements about what state change is, culminating in one sentence — every change of state is a guarded, staged, recorded, scoreable movement of quantities between addresses, interpreted by a single closed engine. The primitive set is engineering: a concrete set of record shapes, presented in full, that realizes the principles and is directly implementable by others. The rules are the test: thirteen falsifiable rules, paralleling Codd's, that determine whether an arbitrary primitive set genuinely realizes the principles rather than realizing them in name. The paper closes by demonstrating that the same primitive set expresses domains as distant as enterprise resource planning, agent simulation at the depth of Dwarf Fortress, and declarative infrastructure orchestration, because under the theory a domain is an assignment of meaning to records, not a body of code.

Part I — The Theory

1. The problem: behavior was never normalized

The relational model succeeded for a structural reason. Before it, every application owned its file formats and the access code that read them; facts were stored redundantly, and update anomalies were a permanent class of defect. Codd eliminated the redundancy of fact: each fact stored once, addressed by key, reachable by one algebra. Per-application data code largely died, and an entire defect class died with it.

No equivalent event occurred for state change. The standard method of software development is verb manufacture: model the domain as nouns, then write one bespoke function per behavior. Each function decides its own validity, applies its own mutation, and records or fails to record its own history. A program is an ever-growing collection of hand-made verbs, and every new feature adds more, because the method offers no alternative.

The founding observation of this theory is that nearly every behavior is the same behavior. Quantities move between addressed locations, under conditions, on a schedule, by preference. What differs between domains is vocabulary, and vocabulary can be data. Therefore behavior can be data, and the verb count of a system can be one.

This observation eliminates a redundancy exactly symmetric to Codd's. Where his normalization removed the redundancy of fact, this normalization removes the redundancy of verb: each kind of change is defined once, addressed by id, and referenced from every site that applies it. The defect class that dies is the behavior anomaly — the duplicated check, the bypassed validation, the unaudited write, the AI that chooses actions under a different rule set than the one that governs their execution. These become structurally impossible for the same reason update anomalies became impossible in normalized data: the redundant copies that could disagree no longer exist.

2. The twelve principles

The principles are stated in dependency order. Each builds on the ones before it, and the final principle is the closure property that the rest jointly imply.

Principle 1 — All state is quantity at an address. Every fact in a system reduces to a number, of a named kind, at a reachable location. There are no privileged datatypes that carry meaning. A health point, a priority, a status, a stock level, a replica count, and a reference to a renderable image are all the same shape: a value belonging to a named group, held at an addressable location.

Principle 2 — All change is movement between addresses. There is exactly one primitive act: quantities move from one address to another, appear at an address (a source), or vanish from an address (a sink). Every behavior, however complex it appears at the domain level, decomposes into movements.

Principle 3 — Change is guarded. No movement occurs unconditionally. Every movement carries its preconditions with it, as data, in the same record: costs that must be

payable, requirements that must hold. Validity is a property of the change record itself, never a property of the code path that requested the change.

Principle 4 — Change is staged, delivered, and atomic. Deciding a change and applying it are different moments. Changes are packaged, delivered in known windows, and applied or refused as whole sets, never partially. The point of decision writes an intent; a later pass delivers it.

Principle 5 — Change is recorded, including refused change. The history of the system is an append-only ledger of every attempted movement, carrying its phase, its order, its provenance, and — for refused movements — the reason for refusal. The audit trail is not instrumentation added to behavior. It is a structural consequence of all behavior being one verb: since every change passes through one executor, recording is a property of the executor, and nothing can change state without leaving a record.

Principle 6 — Derivation is composition of readings. Computed values are not code. They are records that name which addresses to read and how to scale, bound, curve, and combine the readings. An equation is rows, and because it is rows, it is inspectable, editable, and referenceable like all other rows.

Principle 7 — Classification is derivation with a name. Discrete states — statuses, conditions, titles, phases of life — are not a separate system. They are guarded writes of label values, evaluated in a defined order with first match winning. A label, once written, is itself a quantity at an address, available as input to every other principle: a requirement can demand it, a calculation can read it, a preference can weigh it.

Principle 8 — Preference is derivation over candidate changes. Choosing is not a separate faculty from changing. A decision is a set of candidate movements, each scored by reading current state through configured ranges, curves, and weights, with the highest score selected. An economy that can only validate movements is a database. An economy that can rank candidate movements and select one is an agent. The decision layer is therefore not a system beside the economy; it is the economy evaluating itself, and it can never disagree with the rules it acts under, because the rules and the preferences are fields of the same record.

Principle 9 — Meaning is assignment, not structure. The interpreter knows no domain concepts. “Job,” “hobby,” “invoice,” “pod count” exist only as name records attached to addresses. Because no meaning is welded into the interpreter, any domain fits, domains coexist without interference, and one running state supports many interpretations.

Principle 10 — Absence is a value, and it is inert. Every principle above must degrade to nothing gracefully. An unset value, an unassigned reference, and an empty candidate list are ordinary data that the interpreter skips — never a gate, never an error. This is what makes systems authorable while running: a half-built behavior is not a broken behavior, it is a set of records whose absent parts do nothing yet.

Principle 11 — The interpreter is single and closed. One validator decides all validity. One executor applies all change. One scorer scores all preference. One derivation engine composes all calculation. One classification pass writes all labels. And there is no path

around them, including for the system's own internal phases, which are consumers of the same functions. This principle is not an implementation preference. It is what makes Principles 1 through 10 true everywhere rather than true where someone remembered to follow them. A second validator is a place where validity can disagree with itself; a path around the executor is a change the ledger never saw. Closure is the difference between a property and a habit.

Principle 12 — Therefore: new behavior is new rows, never new verbs. This is the closure property and the theory's test. If Principles 1 through 11 hold, then any new domain is expressible as records against the existing shapes, and the correct response to every feature request is data. The day a new problem cannot be answered with data is the day the realization — not the theory — is shown incomplete.

3. The theory in one sentence

Every change of state is a guarded, staged, recorded, scoreable movement of quantities between addresses, interpreted by a single closed engine.

Each word of the sentence is one principle load-bearing: guarded (3), staged (4), recorded (5), scoreable (8), movement (2), quantities (1), addresses (1, 2), single closed engine (11). Derivation (6), classification (7), meaning-as-assignment (9), and inert absence (10) are what make the sentence sufficient in practice rather than merely true, and Principle 12 is what the sentence implies when held everywhere.

4. The status of the three claims

The paper's claims are of three kinds, and stating which is which is necessary for the claims to be evaluated honestly.

The principles are theory. They assert what state change is, in general, in any system whose state is quantities and whose dynamics are movements of quantities. They are argued from the structure of the problem, and they stand or fall on whether counterexamples exist: a behavior that does not decompose into guarded, scheduled, scored movement.

The primitive set is engineering. Codd's model was deducible from first-order logic; that was its particular power, and this theory does not claim the same power. The concrete record shapes presented in Part II — the specific decomposition into groups, values, links, conversions, sets, envelopes, transactions, calculations, labels, considerations, and goals — are not deducible from the principles. Many primitive sets could realize the twelve principles; this one is presented because it exists, runs, and is complete enough to copy. Its historical origin is a long sequence of discarded foundations, including whole subsystems built, run, and decommissioned; that history explains why the shapes are what they are, but it is history, not requirement. A reader implements from this paper, which is a document the original process never had.

The rules are the test. The thirteen rules of Part II are the bridge between the two: they determine whether an arbitrary primitive set — this one or any other — genuinely realizes the principles rather than realizing them in name. Codd wrote his rules because vendors

shipped “relational” systems that were not; the analogous dilution here is the “data-driven” engine that is configuration over welded verbs. The rules are written to be failed.

Part II — The Concrete Realization

5. The thirteen rules

Codd’s twelve rules defined when a system is *genuinely relational* rather than relational in name — a test against dilution. These are the equivalent: the rules that define when a system is genuinely a normalized-behavior interpreter, written from what the engine actually enforces. Codd’s Rule 0 has its counterpart first.

Rule 0 — The Interpreter Rule.

The system must run every game capability through the primitives and their interpreter alone. If any authored behavior requires code, the system does not qualify. (Codd’s Rule 0: the system must manage the database entirely through its relational capabilities.)

Rule 1 — The Record Rule.

All meaning in the system — state, rule, derivation, classification, identity, taxonomy — is represented exactly one way: as records of the fixed shapes. A group is a name. A value is a number of a group. A conversion set is condition, mutation, preference. Nothing meaningful exists outside records. (Codd 1: all information is represented as values in tables.)

Rule 2 — Guaranteed Address.

Every capability is reachable by id: group id, set id, calculation id, goal id, entity id. Any struct that gains an id field of the right kind gains everything the id reaches. Addressing is the whole coupling mechanism; there is no other. (Codd 2: every datum is logically accessible by table name, key, and column.)

Rule 3 — Systematic Absence.

Absence is uniform and inert: `-1` for ids, `ResourceUnset` for floats, tested by exact equality. An absent thing is ignored, never a gate, never an error. Unauthored features do nothing; partially authored games run at every stage of construction. (Codd 3: systematic null handling, independent of data type.)

Rule 4 — The Self-Describing System.

The system’s own vocabulary lives in the same records it interprets. The group table describes the groups; built-in meanings (Upkeep, Sweep, Job, Animation) enter as seeded records in the same table authors extend; the editor browses live structs, not exports. The catalog *is* the database. (Codd 4: the catalog is relational and queryable like the data.)

Rule 5 — The One-Interpreter Rule.

One validator decides all validity. One executor applies all change. One scorer scores all preference. One calculation engine composes all derivation. One classification pass writes all labels. Any consumer needing these calls the same implementation; a second implementation of any decision disqualifies the system. (Codd 5: one comprehensive language governing all definition and manipulation.)

Rule 6 — Live Projection.

Every view of the system — prop panel, debugger, overhead stats, field-replaced dashboard — is a projection of live records, and writes through a projection land in the records. What the simulation means is itself assigned by records (replacement sets), so any running state supports any interpretation. (Codd 6: updatable views.)

Rule 7 — Staged, Set-Level Change.

No state changes at the point of decision. Every change is staged as an envelope, delivered in a known window, validated as a whole set, applied as a whole set, and recorded as one transaction — including refused ones. Work happens on the trigger's frame; delivery happens on a later pass. (Codd 7: set-level insert, update, delete.)

Rule 8 — Authoring Independence.

Records survive the engine's growth. New fields default to absent and old records gain capabilities without editing; superseded fields remain because records serialize them; membership is by reference, so new content joins existing rules (parent expansion, auto-selection) without touching them. (Codd 8: physical data independence.)

Rule 9 — Semantic Independence.

The interpreter knows no game meaning. Every leaf is a number; enums carrying authored meaning are migrations in progress, not architecture. Because no meaning is welded in, all meaning is reassignable — the same simulation is a colony, a battle, or a ledger of accounts by pointing ids elsewhere. (Codd 9: logical data independence.)

Rule 10 — Rules Live in Records, Not in Callers.

Every condition, cost, preference, classification, and derivation is stored once, in its own record, referenced by id from every site that applies it — never copied into callers, never inlined into consumers. Changing a rule is editing one record; nothing else moves. (Codd 10: integrity constraints stored in the catalog, not in application programs.)

Rule 11 — Distribution Independence.

A scene is complete and self-describing: copying it is saving it; restoring is copying back; shadow scenes replicate it; scene sets compose scenes into apps with path-scoped read and write. Records do not know or care where they run. (Codd 11: distribution independence.)

Rule 12 — The Nonsubversion Rule.

There is no lower path around the interpreter. No code may move a value except through the executor, decide validity except through the validator, or score except through the scorer — including the engine's own phases, which are consumers of the same functions.

Adding a domain never adds a primitive: the answer to every new problem is a field, and the day that fails is the day this rule is tested. (Codd 12: no bypassing the integrity rules through a lower-level interface.)

Codd wrote his twelve because vendors were shipping “relational” systems that weren’t, and he needed a falsifiable test. These serve the same function against the analogous dilution — “data-driven” engines that are configuration over welded verbs. Rules 0, 5, and 12 are the ones such systems always fail: content requires code, decisions have parallel implementations, and there is always a lower path.

6. The primitive set

The shapes below are the realization’s complete behavioral vocabulary, given in the engineering language of the running system (Zig). Each principle of Part I maps onto named structs; the full source appears in the appendix. A reader building a realization in another language needs the shapes and the rules; nothing else in the implementation is theoretically significant.

Quantity at an address (Principle 1). `ResourceValue` is the leaf of the entire system: a group id and an `f32` value, with optional limits, a limit curve, an optional calculation reference with per-frame caching, and a stack size. `ResourceGroup` carries the names: id, name, icon reference, display metadata, translations. `ResourceLink` is the uniform address: a group, a link type, an optional data path, an optional record id, an optional literal. Everything the interpreter reads or writes, it reads or writes through these three shapes.

Movement (Principle 2). `ResourceConversion` is the one verb in record form: a `from` link and a `to` link, values for each side, a set-versus-add flag, parent scaling references, a response curve, and directional constraints. Nothing else in the system mutates state.

Guards (Principle 3). `ResourceConversionSet` aggregates conversions into the unit of validity: `costs` (movements that must be payable) and `requirements` (conditions that must hold), validated as a whole set.

Staging (Principle 4). `ResourceEnvelope` is the staged intent: owner, targets, the conversion set to apply, instant or durational delivery expressed in both real seconds and turns, a scale, a forced flag, success and failure commands, and actor-creation fields processed in a pre-pass so that created entities exist before entity logic runs. `ResourceTrigger` polls a condition and fires an event; `ResourceSequence` orders conversions through time; a single constant, `RealTimeTurnInSeconds`, converts between turn-based and real-time interpretations of every duration in the system.

The ledger (Principle 5). `ResourceTransaction` records every attempted application: source, target, conversion set, validity, the indices of failed costs and requirements, the phase of the turn that produced it, a monotonic order, the first failing reason, and whether it was built-and-inspected or built-and-submitted. The ledger is append-only with a walk marker; refused transactions are recorded with the same fidelity as applied ones.

Derivation (Principle 6). `ResourceCalculation` composes readings in four lists — base, base scale, modifier, modifier scale — of `ResourceCalculationItem`: a group to read, optional randomness, optional inventory-slot filtering, a scale, limits with curves, inversion for subtraction and division, modulo, live-value clamps (a limit that is itself a group reference rather than a literal), and player-wide summation. Results cache per frame through the `ResourceValue` calculation fields.

Classification (Principle 7). `ResourceLabel` evaluates a conversion set’s requirements against an entity and, on pass, writes a label integer to a target link. Multiple labels sharing a target form an ordered switch, first match wins, no match writes absence. The label value is itself a resource value, queryable and gateable everywhere. The realization contains no other status system, because this shape is the status system.

Preference (Principle 8). `ResourceConsideration` — a calculation reference, a range, a weight, a curve, inversion, zero ranges, and floor values — is embedded directly in `ResourceConversionSet`, making every set a scoreable behavior candidate by construction. `ResourceGoal` extends preference to purpose: an activation calculation, spatial selection through heat-map items, target selection over actors and tiles, and an outcome coupling animation events to conversion sets.

Meaning as assignment (Principle 9). Conversion sets carry a `resource_group_id` as their classifier, so behavior domains are distinguished only by data. The system’s own built-in meanings are seeded rows in the same group table that authors extend.

Inert absence (Principle 10). `ResourceUnset` is defined as the most negative `f32`; ids use `-1`; both are tested by exact equality through one function. Whether an unset input voids a whole calculation is itself an authored flag (`any_unset_returns_unset`), making strictness a per-record choice rather than an engine policy.

The closed interpreter (Principle 11) and closure (Principle 12) are not structs; they are the discipline the rules test, and the next section shows what that discipline costs at the margin.

7. The cost model: one domain, one field

The theory’s economic claim is that under full compliance, the marginal cost of a behavioral domain collapses to naming it. The realization’s job system is the demonstration.

A worker-and-priority job system of the kind found in colony simulations conventionally requires a job data type, a work-discovery hierarchy, a priority model, a scheduler with reservation and interruption handling, a priority UI, and integration code in every system jobs touch. It is a subsystem, built over weeks to months, and it permanently enlarges the engine’s surface.

In the realization, the complete engine-side addition was one field on the actor:

```
// If !=-1, this points to the Resource Conversion Set which has UAI
```



```
// considerations for our Jobs.  Make a `Jobs` parent group, add Groups
// as jobs, put in a set with UAI considerations, score.  Per actor jobs
job_resource_conversion_set_id: i32 = -1,
```

The comment is the entire authoring recipe, and every capability it invokes already existed: jobs enter as vocabulary (Rule 4), each job is a conversion set whose considerations score it (Principle 8), the one scorer selects among candidates (Rule 5), per-actor priority is a resource value read by one calculation item, and the player's priority control and the AI's decision input are the same number — one value, two writers, one reader, incapable of disagreement. Job gating by condition is a label requirement; job effects over time are envelopes; every job decision, taken or refused, lands in the transaction ledger. The domain's genuinely new information content was one fact — *this actor has a set of scoreable work options* — and one fact is exactly one id wide.

The field was later generalized:

```
// If items are !=-1, this points to the Resource Conversion Set which has
// UAI considerations for our Jobs.

//NOTE: Standard Case: Make a `Jobs` parent group, add Groups as jobs, put
// in a set with UAI considerations, score.  Per actor jobs

//NOTE: This is a list, because this is the place I can add N different
// sets of behavior scorers, using the

// ResourceConversionSet.resource_group_id to classify and group them

// (Jobs, Hobbies, whatever)

job_resource_conversion_set_id: []i32 = &.{},
```

One character of type signature, `i32` to `[]i32`, converted a named feature into a hosting slot for unlimited behavior domains, classified only by the sets' own group ids. The engine no longer knows what the domains are; a game with no jobs and seven other domains uses the same field. This is Principle 12 exhibited twice at its real price: the first domain cost one field, and every subsequent domain costs zero fields.

8. Domain optimizations: fast paths beside the general path

Full semantic independence does not forbid fixed vocabulary; it forbids fixed vocabulary as the *only* vocabulary. A realization serving high-volume authoring will correctly grow

domain optimizations: fixed-name fast paths for the most frequent operations, placed beside the general id path that covers everything else.

The realization's animation frames carry both. Each frame holds a `Conversion Set` id — the general path, reaching the entire behavior system — and five one-click flags (`Footstep`, `Airborne`, `Cancelable`, `Invincible`, `Spawn`) covering the five markings that appear on nearly every animated actor. The flags are cheap to offer precisely because their meanings are already interpreter-reachable concepts; each is one boolean where a conventional engine would need an event hook and a callback contract. The test for whether such a path is a legitimate optimization or a Rule 9 violation is single: does a general path exist beside it? Fixed vocabulary beside an id slot is a keyboard shortcut; fixed vocabulary alone is a ceiling.

9. Three domains, one interpreter

The theory claims that a domain is an assignment of meaning to records. The claim is best tested against domains far apart from one another. Three follow, chosen because each is normally considered a distinct discipline with its own engineering tradition.

9.1 Enterprise resource planning (SAP-class systems)

An ERP system is already an economy interpreter, which makes the mapping nearly an identity rather than a translation. Materials movements are conversions between addressed stores. Cost-center postings are conversions whose links carry account groups. Condition-checked postings are conversion sets: the requirements are the posting rules, the costs are the movements, and refusal-with-reason is the transaction ledger recording a failed set. Batch jobs delivered in posting windows are envelopes with delivery timing. Document flow — the audit chain from order to delivery to invoice — is the append-only transaction ledger, obtained structurally rather than built as a compliance feature. Pricing procedures, which in practice are ordered condition tables with access sequences, are calculations: base, scales, modifiers, and curves over addressed values.

The mapping also predicts ERP's characteristic failure mode. Real ERP deployments degrade through custom code that bypasses the configured layer — user exits and custom programs that post directly. Under the rules, that is a named violation: Rule 12, the lower path. The theory thus does not merely express the domain; it explains where the domain's existing systems rot, and the explanation is that they are partial realizations without the nonsubversion property.

9.2 Deep agent simulation (Dwarf Fortress-class behavior)

The behavioral layer of the deepest agent simulations decomposes onto the primitive set with little remainder. Labors are a parent group of jobs: requirements gate eligibility (tool possession, capability labels), considerations score assignment (skill values, distance calculations), and priority overhauls are one more consideration reading a priority group. Needs — eat, drink, pray, socialize — are a second domain in the same hosting slot: deprivation values incremented by upkeep conversions, urgency shaped by response curves.

Personality facets and preferences are per-actor values scaling considerations, so an agent who loves craftsmanship scores crafting higher through one calculation item. Emotional state is the label switch: bands over an accumulated stress value writing Content, Stressed, Haggard, Berserk — and because a label is a queryable value, the written state gates every other domain's sets. The famous cascade dynamics of such simulations — a bad event feeding a mood, feeding conflict, feeding worse events — require no cascade engine; they are conversions writing values that flip labels that gate conversion sets, composing at the record level.

The honest boundary: the theory normalizes behavior, not physics or generative content. Fluid mechanics, geology, and procedural world generation are substrates outside the claim — although body models reduce further into the vocabulary than expected, since a limb with a motor value gating a grasp capability is values and conversions again. The precise statement is that the primitive set expresses the part of such simulations that makes agents seem alive, and does not express the part that makes terrain seem real.

9.3 Declarative infrastructure (Kubernetes-class orchestration)

An orchestrator's state is quantities at addresses: replica counts, CPU and memory requests, capacity per node. Desired state is a record; reconciliation is the computation of movements that close the gap between observed and desired quantities — a conversion set whose requirements are the spec's constraints and whose executor is the node agent. Scheduling — the choice of node for a pending pod — is preference: candidate placements scored by reading capacity, affinity, and taint values through weights, highest score selected, which is the scorer operating over candidate movements. Rolling updates with surge and unavailability bounds are staged, set-level change: envelopes delivered in windows, applied or refused as sets. Event streams and audit logs are the transaction ledger. Namespaces are the partition key of Rule 11: complete, self-describing record sets with no channel of interference other than explicit reference.

Here too the mapping predicts the failure mode. The orchestration ecosystem's answer to insufficient vocabulary was the operator pattern — readmitting arbitrary code into the control loop — which under the rules is the Rule 0 failure: content required code because the record vocabulary was not general enough, and no field-shaped answer existed. The theory's claim is that with a sufficient primitive set, the extension mechanism is the one Principle 12 names: new rows, not new controllers.

9.4 What the three mappings jointly show

The three domains share no vocabulary, no tradition, and no tooling, yet each decomposes onto the same twelve shapes, and each domain's known degradation pattern corresponds to a named rule violation. This is the practical meaning of Principle 9: the interpreter that runs a colony simulation is, without modification, the interpreter that posts a materials movement or schedules a pod, because “what domain is this” was never a property of the interpreter. In the realization this is not hypothetical composition: games are partition keys, many games run concurrently in one process with total mutual isolation, and a domain as different as an ERP ledger would enter the same way every game does — as

rows.

10. Adoption: the two regimes

The theory admits two modes of use, and they are different in kind, not degree.

Partial adoption moves conditions, costs, preferences, and classifications out of call sites and into referenced records — partial Rule 1 and Rule 10 — while the system’s features continue to originate in code. This pays immediately and degrades gracefully: each migrated rule is one less duplicated check, and a bypass under deadline costs only that feature’s share of the benefit. This is how the relational model actually spread — almost no one ran a twelve-rule database, but per-application data code died anyway — and it is the transferable result for most teams: less code, more behavior as data, faster features.

Full compliance is authorship: features originate in records, and code originates nothing. It is binary, because its defining property is the absence of any path around the interpreter, and one bypass ends the property entirely. The two regimes are separated by a discontinuity, not a gradient: burden relief continued indefinitely does not become authorship. The rules of Section 5 test the far side only; the near side needs no test, because partial benefit is proportional and self-evident.

11. Conclusion

Codd’s normalization gave computing a uniform theory of data at rest and killed per-application storage code. This paper has stated the symmetric theory for data in motion: twelve principles reducing all state change to one sentence, a concrete primitive set that realizes the principles and is complete enough to implement from, and thirteen rules that test any realization against dilution. The principles are theory; the primitive set is engineering; the rules are the bridge. The cost model under full compliance is the theory’s sharpest practical consequence — one domain, one field; subsequent domains, zero — and the three domain mappings show why: when meaning is assignment rather than structure, a domain was never anything but rows.

Appendices to “The General Theory of State Change”

These appendices support the paper without repeating it. They collect the mappings, correspondences, vocabularies, and boundary analyses that the paper’s argument implies but does not enumerate. Appendix A (full source of `resource_system.zig`) precedes these and is referenced throughout by `struct` and `field` name.

Appendix B — Full Correspondence: Codd’s Rules, the Thirteen Rules, and the Principles

The paper states the rules and the principles separately. This table gives the three-way correspondence, including where the mapping is exact, where it is analogical, and which structs enforce each rule.

Rule	Codd’s original (para-phrase)	This system	Mapping quality	Enforcing shapes / mechanisms	Principle(s) realized
0	Manage the database entirely through relational capabilities	All capability through primitives and interpreter alone	Exact	The interpreter as sole consumer of all records	11, 12
1	All information as values in tables	All meaning as records of fixed shapes	Exact	ResourceGroup, Re-source-Value, Re-source-ConversionSet, all others	10, 11
2	Every datum accessible by table + key + column	Every capability reachable by id	Exact	id fields; Re-sourceLink as universal address	1
3	Systematic nulls independent of type	-1 / Re-source-Unset, exact equality, inert	Exact	isUnset(), Re-source-Value.isUnset(), defaults on every struct	10

Rule	Codd's original (para-phrase)	This system	Mapping quality	Enforcing shapes / mechanisms	Principle(s) realized
4	Catalog is relational, queryable like data	Vocabulary lives in the records it describes	Exact	Seeded groups (Upkeep, Sweep, Job, Animation) in the author-extended group table	9, 1
5	One comprehensive language	One validator, executor, scorer, calculator, classifier	Analogical (language → interpreter functions)	The five singleton functions; disqualification on duplication	11
6	Updatable views	Live projection; writes through views land in records	Exact	Editor panels over live structs; prop panel = debugger = dashboard	1, 11
7	Set-level insert/update/delete	Staged develop, whole-set validate/apply, ledgered	Analogical (set-orientation → atomicity in time)	ResourceEnvelope, Re-source-Transaction, delivery pre-pass	4, 5
8	Physical data independence	Records survive engine growth	Analogical (storage → engine version)	Absent-default new fields; serialized superseded fields; membership by reference	10, 12

Rule	Codd's original (para-phrase)	This system	Mapping quality	Enforcing shapes / mechanisms	Principle(s) realized
9	Logical data independence	Semantic independence; meaning reassignable	Analogical (schema change → meaning change)	Numbers at leaves; group-id classification; enums as migrations	9
10	Integrity constraints in catalog, not applications	Rules in records, referenced by id, never inlined	Exact	ResourceC3,6,7,8 referenced from every application site	3, 6, 7, 8
11	Distribution independence	Scenes complete and self-describing; games as partition keys	Exact	Scene closure; game id; N-game concurrency without shared mutable state	9, 11
12	No subversion via lower-level interface	No lower path; engine phases consume the same functions	Exact	Enforcement history; field-not-primitive answers	11, 12

Reading note. Four rules are analogical rather than exact (5, 7, 8, 9), and in each case the analogy strengthens rather than weakens: Codd's Rule 7 concerns set-orientation in space (many rows at once); Rule 7 here adds set-orientation in *time* (decision and delivery as separate moments), which Codd's model never needed because databases have no frame. The behavioral theory required the staging half that the storage theory could omit.

Appendix C — The Principle-to-Struct Evidence Matrix

The paper maps principles to shapes in prose. This matrix gives the complete cross-reference, including secondary realizations, and identifies which fields carry each principle’s load.

Principle	Primary shape	Load-bearing fields	Secondary realizations
1. Quantity at address	ResourceValue	group, value	ResourceGroup (the names), ResourceLink (the address grammar), ResourceInstance.values (composition)
2. Movement	ResourceConversion	from, to, from value, to value, is set	parent a/b+ scales (movement composing movement); is bidirectional, is only increasing (movement constrained)
3. Guards	ResourceConversionSet	cons, requirements	ResourceTrigger.condition * (guard as poll); ResourceLabel.conversion set id (guard as classifier input)
4. Staging	ResourceEnvelope	is instant, duration seconds/turns, elapsed *	ResourceSequence (staging as ordered series); ResourceTrigger.poll interval * (staging as recurrence); actor-creation pre-pass fields

Principle	Primary shape	Load-bearing fields	Secondary realizations
5. Ledger	ResourceTransaction	phase, order, reason, failed *, indices, is applied, is last	envelope slot, game frame, game time (provenance); is deleted reuse discipline
6. Derivation	ResourceCalculation	base, base scale, modifier, modifier scale	ResourceCalculationItem.invert (subtraction/division), modulo *, limit min/max resource group (live clamps), is player sum (aggregation)
7. Classification	ResourceLabel	label, target, conversion set id	Ordered evaluation, first-match, -1 on no-match; image id (classification driving presentation)
8. Preference	ResourceConsideration	score, resource calculation id, range, score weight, curve	force min value (defaults), zero ranges (exclusions), score inverted (avoidance); ResourceGoal (preference with purpose, space, outcome)
9. Meaning as assignment	ResourceConversionSet	group id	Seeded vocabulary rows; job resource conversion set id: []i32 as unnamed domain host

Principle	Primary shape	Load-bearing fields	Secondary realizations
10. Inert absence	ResourceUnset, — -1		any unset returns unset (strictness as authored choice); every struct's defaults
11. Closed interpreter	(<i>discipline, not struct</i>)	—	Enforcement record: refused ResourceStatus, refused scheduler, refused ResourceJob
12. Rows not verbs	(<i>closure, not struct</i>)	—	The job field; its slice generalization; the 1–3-field feature pattern

Reading note. Principles 11 and 12 have no primary struct by necessity: one is the absence of alternative code paths, and the other is a property of the whole. Their evidence is historical (refusals) and economic (field counts), which is why the rules — not the shapes — are the test for them.

Appendix D — The Refusal Record

The paper cites the enforcement history in one line. This appendix expands it, because each refusal is a worked example of Rule 12 reasoning, and together they demonstrate the decision procedure a maintainer of a realization must apply.

Proposed addition	Conventional justification	Why refused	What answered the need instead	Rules invoked
ResourceStatus struct	“Every game needs statuses: dead, stunned, burning”	A status is a discrete state derived from values — which is classification, which exists	ResourceLabel, ordered first-match switch writing label values; the source comment records the reasoning verbatim (“we dont need ResourceStatus, because that is what Label provides”)	5, 11, 12
Scheduler subsystem	“Timed and recurring behavior needs a scheduler”	Delivery-in-windows already exists as staging; a scheduler would be a second executor with its own clock	ResourceEnv, durations, Re-sourceTrigger polling, Resource-Sequence advancement, RealTime-TurnIn-Seconds as the single time conversion	5, 17, 18, 19, 20
ResourceJob struct	“Jobs are a major feature; major features get types”	A job is a scoreable conversion set; a job <i>system</i> is a domain of them; domains are rows	job resource conversion set id (one field), later generalized to [] i32 (zero further fields per domain)	9, 12

Proposed addition	Conventional justification	Why refused	What answered the need instead	Rules invoked
Fact/inference layer (built, then decommissioned)	“Complex conditions need logical inference”	Not refused in advance — built, run, and found redundant: requirements over values plus labels-as-values covered the load	ResourceConservationSet.requirements + Resource-Label composition; decommissioned fields remain serialized per Rule 8	8, 12 (from set, respectively)

Reading note. The fourth row is different in kind and included deliberately: it shows the rules operating on the system’s own past, not only on proposals. A realization is permitted wrong subsystems; Rule 8 is what makes them survivable (their records persist inertly), and Rule 12 is what eventually identifies them as redundant. The refusal discipline is not prescience. It is the willingness to ask “which existing principle already covers this” *before* adding, and “which existing principle now covers this” *after*.

Appendix E — The Diagnosis Procedure

The paper’s cost model (“one domain, one field”) presupposes a step it does not describe: classifying an incoming feature request into the primitive vocabulary. This appendix states that procedure. It is the practical skill of operating a realization, and it is teachable.

The five questions, asked in order:

#	Question	If yes, the feature is a...	Realizing shapes
1	Is this a choice among options?	Scoring problem	Conversion sets with considerations; the scorer; force min value for defaults
2	Is this about <i>where</i> — location, territory, attraction, avoidance?	Spatial problem	Heat-map items, Resource-GoalTarget, zones
3	Is this a named condition an entity is <i>in</i> ?	State problem	Labels: bands over values, ordered switch

#	Question	If yes, the feature is a...	Realizing shapes
4	Is this a number computed from other numbers?	Derivation problem	Calculations: base/modifier lists, curves, clamps
5	Is this something that happens <i>later</i> , <i>over time</i> , or <i>when a condition holds</i> ?	Timing problem	Envelopes, triggers, sequences

Composite features decompose in this order. “Actors have favorite spots they visit when idle” = state (an OffDuty label, Q3) + spatial (a per-actor heat map, Q2) + scoring (an idle-domain conversion set whose consideration reads the label, Q1). Each component is 1–3 fields of authored data; none is code.

The procedure’s halting condition is Rule 12’s test. If all five questions return no — the request is not choice, place, state, derivation, or timing — then either the request is outside the theory’s scope (see Appendix H) or the realization’s primitive set is incomplete, and the day has arrived that Rule 12 names. The procedure thus doubles as the falsification protocol: it is how a maintainer discovers, feature by feature, whether closure holds.

Appendix F — Extended Domain Mapping Tables

Section 9 of the paper argues three domains in prose. These tables give the term-by-term mappings, plus two domains the paper did not include (email infrastructure; the spreadsheet), to widen the evidence that meaning is assignment.

F.1 Enterprise resource planning

ERP concept	Theory concept	Shapes
Material movement	Conversion	ResourceConversion
Posting rules / validation	Guards	costs, requirements
Cost center / GL account	Group	ResourceGroup
Pricing procedure (condition tables, access sequences)	Derivation	ResourceCalculation with ordered items, scales, curves
Batch posting window	Staged delivery	ResourceEnvelope durations
Document flow / audit chain	Ledger	ResourceTransaction with phase, order, provenance

ERP concept	Theory concept	Shapes
Refused posting with error	Refused set with reason	<code>is valid=false,</code> <code>reason, failed *</code> <code>indices</code>
Company code isolation User exit / custom ABAP posting directly	Partition key Rule 12 violation	<code>game id</code> <i>(the named degradation)</i>

F.2 Deep agent simulation

Simulation concept	Theory concept	Shapes
Labor / work type	Domain of scoreable sets	Sets under a <code>Jobs</code> parent group
Labor priority	Value read by consideration	Priority group + one <code>ResourceCalculationItem</code>
Need (eat, pray, socialize)	Deprivation value + urgency curve	Upkeep conversions + consideration curves
Personality facet / preference	Per-actor scaling value	Facet group scaling considerations
Mood / emotional state	Label band over accumulated value	<code>ResourceLabel</code> switch on a stress group
Tantrum cascade	Record-level composition	Conversions → values → labels → gates, no cascade engine
Military alert / schedule switching	Activation calculation or label-gated sets	<code>is active</code> <code>resource</code> <code>calculation id</code>
Uniform / equipment	Slot-gated instance	<code>inventory slot</code> <code>resource group id, equipped id</code>
Reservation (two workers, one job)	Whole-set validation	Set-level apply/refuse; ledger ordering

F.3 Declarative infrastructure

Orchestration concept	Theory concept	Shapes
Replica count, resource request, capacity	Quantities at addresses	<code>ResourceValue</code>
Desired-state spec	Guard set	<code>requirements</code>

Orchestration concept	Theory concept	Shapes
Reconciliation loop	Gap-closing conversions	Conversions computed against observed vs. desired
Scheduler (pod placement)	Preference over candidates	Considerations reading capacity/affinity values
Rolling update (maxSurge / maxUnavailable)	Staged, bounded set delivery	Envelopes with delivery windows
Event stream / audit log	Ledger	<code>ResourceTransaction</code>
Namespace	Partition key	<code>game id / scene closure</code>
Health check → status	Classification	Labels (Ready, Degraded, Failed as bands)
Operator pattern (arbitrary code in the loop)	Rule 0 failure	<i>(the named degradation)</i>

F.4 Email infrastructure (*not in the paper*)

Included because one struct maps so directly that the domain is nearly self-demonstrating.

Email concept	Theory concept	Shapes
Message	Staged intent with addressing	<code>ResourceEnvelope: owner entity id, target entity ids</code>
Queued delivery	Delivery window	<code>is instant=false, durations</code>
Delivery receipt / bounce	Success/failure paths	<code>command success, command failure</code>
Accept-whole-or-reject-whole (transactional delivery)	Atomic set application	Rule 7
Mail log	Ledger	<code>ResourceTransaction</code>
Attachment spawning content	Creation via delivery	<code>create from entity id pre-pass</code>

F.5 The spreadsheet (*not in the paper*)

Included because it is the mass-market system closest to a partial realization, and the comparison locates exactly what it lacks.

Spreadsheet concept	Theory concept	Present in spreadsheets?
Cell	Quantity at address	Yes — the closest existing relative of <code>ResourceValue</code>
Formula	Derivation as record	Yes — equations as data, one evaluation engine
Recalculation	Frame-cached derivation	Yes — analogous to resource calculation game frame
Guarded mutation	Guards	No — any cell writable at any time, unconditionally
Staged change	Staging	No — edits apply at the point of decision
Transaction ledger	Ledger	No — undo stack is not a queryable history
Preference / choice	Scoring	No — a spreadsheet derives but never decides

Reading note. The spreadsheet row-by-row absence list is the theory’s own genealogy stated negatively: the spreadsheet is Principles 1, 2, 6, and 11 without 3, 4, 5, 7, and 8. Its universal success on four principles, across every industry, is independent evidence for the principles it does realize; its universal failure mode — the unauditable, unvalidated corporate spreadsheet — is the predicted cost of the principles it omits.

Appendix G — The Time Model

The paper mentions `RealTimeTurnInSeconds` once. The time model deserves its own statement, because it is where this theory extends past its relational parent in a way Codd’s domain never required.

The dual clock. Every duration in the system exists in two denominations — turns (`duration_turns`, `elapsed_turns`) and seconds (`duration_seconds`, `elapsed_seconds`) — with one global conversion constant. A single authored behavior is therefore playable in a turn-based game and a real-time game without modification: the envelope that delivers in 3 turns delivers in $3 \times \text{RealTimeTurnInSeconds}$ seconds, and tuning the constant retunes every timed behavior in the game at once.

Game time versus wall time. `updated_game_time` and `game_time` advance through delta time, not wall clock: pausing freezes them. Every timestamp in the ledger and every change-detection field is therefore simulation-consistent — a paused game accumulates no spurious history.

Change detection as a value property. `ResourceValue.setValue` and `set-Bool` update the timestamp only when the value actually changes; writing the same value is not an update. “Time since this changed” becomes a queryable quantity on every value in the system, for free, which is what triggers poll against and what considerations can read (recency as a scoring input).

The three timing shapes and their division of labor. Envelopes answer *deliver this, then* (single future application). Triggers answer *whenever this holds, fire* (recurring condition polls with frame or second intervals). Sequences answer *these, in order* (indexed conversion series with optional time advancement and looping). The refusal of a scheduler subsystem (Appendix D) is the claim that these three shapes exhaust the timing vocabulary a behavioral system needs.

Appendix H — Scope Boundary: What the Theory Does Not Normalize

Section 9.2 of the paper draws the boundary in one sentence. This appendix draws it precisely, because a theory’s value depends on its edges being honest.

Category	Inside or outside	Reasoning
Agent decision (choice among actions)	Inside	Preference over candidate movements — the theory’s core
Economies, inventories, equipment	Inside	Quantities, movements, guards, slots
Status/condition systems	Inside	Classification (labels)
Derived stats of any complexity	Inside	Calculations compose without limit
Timed, recurring, sequenced behavior	Inside	The three timing shapes
Body/combat models (parts, capabilities, damage flow)	Inside, further than expected	A limb is values (Motor, Grasp); wounds are conversions; incapacities are labels
Rendering, audio playback	Outside, but <i>addressed</i> from inside	<code>content import id</code> as a value means records decide <i>what</i> renders; the rasterizer itself is substrate
Pathfinding execution	Outside, but <i>directed</i> from inside	Goals and heat maps choose destinations; the path search is substrate

Category	Inside or outside	Reasoning
Fluid, temperature, structural physics	Outside	Continuous field dynamics, not quantity movement between named addresses
Procedural generation (worlds, histories)	Outside	Content creation, not state change of existing content
The interpreter's own implementation	Outside by definition	Rule 0 governs authored behavior; the interpreter is the one place code is code

The boundary's principle. Everything inside shares one property: its state is *discrete quantities at named addresses* and its dynamics are *movements chosen and guarded*. What falls outside is either continuous-field simulation (physics), content creation (generation), or presentation execution (rendering, path search) — and in each outside case, the theory still holds the *interface*: records decide what to render, where to path, and what generated content becomes, even though they do not perform the rendering, the search, or the generation. The theory normalizes behavior; it delegates substrate, and the delegation points are themselves ids.

Appendix I — Glossary of the Theory's Vocabulary

For readers implementing from this paper. Terms are defined by their role in the theory, with the realization's name in parentheses.

Group (`ResourceGroup`) — A name given to a kind of quantity. The atom of vocabulary. Groups form two-level taxonomies (parents and children), and a parent's membership is a queryable fact.

Value (`ResourceValue`) — A quantity of a group, at an address, optionally bounded, curved, derived, and stacked. The atom of state.

Link (`ResourceLink`) — The universal address: group, link type, optional path, optional record id, optional literal. Anything readable or writable is reachable as a link.

Conversion (`ResourceConversion`) — One movement: from-link, to-link, amounts, mode (add or set), constraints, curve. The atom of change.

Set (`ResourceConversionSet`) — The unit of validity and of behavior: costs, requirements, considerations, and classification (its own group id). The theory's central composite: a set is simultaneously a rule (guards), an action (movements), and a candidate (scores).

Envelope (`ResourceEnvelope`) — A staged intent: who, to whom, which set, delivered when, at what scale, with what follow-up. The atom of scheduled change.

Transaction (`ResourceTransaction`) — One ledger entry: which set, between whom, in which phase, in what order, valid or refused and why. The atom of history.

Calculation (`ResourceCalculation / Item`) — A derivation as data: readings scaled, bounded, curved, inverted, combined. The atom of computation.

Label (`ResourceLabel`) — A guarded write of a discrete state, in an ordered first-match switch. The atom of classification; also the entirety of the status system.

Consideration (`ResourceConsideration`) — One scoring input: a calculation through a range, weight, and curve. The atom of preference.

Goal (`ResourceGoal`) — Preference with purpose: activation, spatial selection, targeting, outcome. The composite that turns scoring into directed agency.

Instance (`ResourceInstance`) — A set realized on an entity: copied defaults, live values, slot membership, equip state. The atom of possession.

Sentinel absence (`-1, ResourceUnset`) — The one representation of “not authored,” inert everywhere.

Domain — Not a shape. An assignment of meaning: a family of sets sharing a classifying group. The theory’s claim is that this row of the glossary never becomes a struct.

Appendix J — Failure-Mode Diagnostic

A companion to the rules for evaluating existing systems. Each row names a common architecture, the rule it characteristically fails, and the observable symptom — usable as a checklist against any system claiming to be data-driven.

Architecture	Characteristically fails	Observable symptom
Config-file-driven engine (“data-driven” in name)	Rule 0	Every config key maps to a hand-written verb; new behavior requires a new key <i>and</i> new code
Entity-component-system	Rule 0, 5	Components are records, but every System is bespoke code; N systems = N private validators
Scripting-layer engine (embedded language)	Rule 0, 12	Content is code in a second language; the script API is the lower path
Business rules engine	Rule 6, 12	Rules are records, but the surrounding application moves state directly; no live projection

Architecture	Characteristically fails	Observable symptom
Event-sourced application	Rule 5, 10	The ledger exists, but each event handler privately validates and applies; rules live in handlers
ERP with custom exits	Rule 12	Direct postings bypass the configured layer; the audit chain has gaps
Orchestrator with operators	Rule 0	Vocabulary exhausted; arbitrary code readmitted to the control loop
Spreadsheet	Rules 3 (partially), 7, 10	Unguarded writes, no staging, no queryable history

Reading note. No row fails Rules 1–4 badly: the record half of the theory is widely and independently discovered. Every row fails on the interpreter half — 0, 5, or 12. This asymmetry, visible across the whole industry, is the paper’s Section 1 claim in diagnostic form: computing normalized its nouns and never its verbs, and the failures are exactly where the verbs are.

[[HOWL-INFO-17-2026](#)] The General Theory of State Change. Github: <https://github.com/ghowland/papers/tree/main/papers/INFO-17-2026>

[[HOWL-COMP-1-2026](#)] Implementing a Tall-Infra Data-Only Execution System . Github: <https://github.com/ghowland/papers/tree/main/papers/COMP/HOWL-COMP-1-2026>

[[HOWL-COMP-12-2026](#)] Closed Loop Architecture. Github: <https://github.com/ghowland/papers/tree/main/papers/COMP/HOWL-COMP-12-2026>

[[HOWL-INFO-11-2026](#)] The Relationship of Zero, One, and Infinity in Information Processing. Github: <https://github.com/ghowland/papers/tree/main/papers/INFO/HOWL-INFO-11-2026>

[[HOWL-INFO-13-2026](#)] The Six States of Information. Github: <https://github.com/ghowland/papers/tree/main/papers/INFO/HOWL-INFO-13-2026>

[[HOWL-MATH-15-2026](#)] A Measurement Theory of Processing. Github: <https://github.com/ghowland/papers/tree/main/papers/MATH/HOWL-MATH-15-2026>

[[HOWL-MATH-20-2026](#)] Derivability Classes of Optimal Reduction. Github: <https://github.com/ghowland/papers/tree/main/papers/MATH/HOWL-MATH-20-2026>

[[HOWL-INFO-14-2026](#)] Bits and Ops. Github: <https://github.com/ghowland/papers/tree/main/papers/INFO/HOWL-INFO-14-2026>

[[HOWL-INFO-15-2026](#)] Cross-Domain Learning: How Bits and Ops Transfer Between Domains. Github: <https://github.com/ghowland/papers/tree/main/papers/INFO/HOWL-INFO-15-2026>

[[HOWL-INFO-16-2026](#)] Staged Advancement: How Dissolved Knowledge Generates the Next Domain. Github: <https://github.com/ghowland/papers/tree/main/papers/INFO/HOWL-INFO-16-2026>